Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 4

Belo Horizonte
29 de abril de 2026

SUMÁRIO

1 EXERCÍCIO 5.7 3

ANEXO A CÓDIGO EXERCÍCIO 5.7 5

1 EXERCÍCIO 5.7

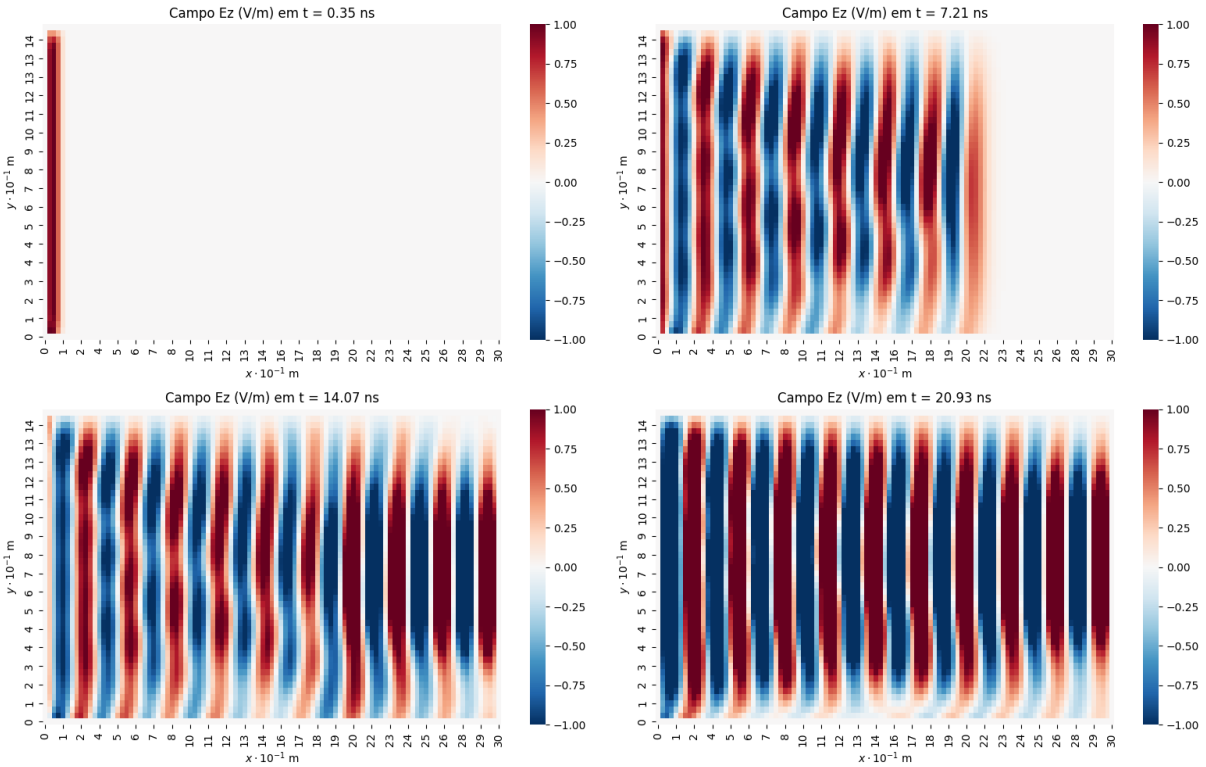
Todo o código usado nesse exercício encontra-se no Anexo A.

O primeiro passo é obter uma simulação estável na geometria proposta pelo problema. Considerando uma fonte de onda planar senoidal com $f = 1$ GHz no espaço livre, temos $\lambda = c/f = 0.3$ m. Usando os seguintes parâmetros de discretização:

- $N_x = 500 \quad , \quad N_y = 100$
- $\Delta y = \Delta x = 0.1 \cdot \lambda = 0.03$ m
- $\Delta t = \frac{0.99}{c\sqrt{(\Delta x)^{-2}+(\Delta y)^{-2}}} = 70$ ps (condição CFL 2D)
- $N_t = 500$

obtemos a seguinte propagação da onda planar a partir da extremidade esquerda do grid, exibida na Figura 1.

Figura 1 – Propagação da onda planar senoidal a partir da extremidade esquerda.



Note que ao atingir a borda direita do grid temos reflexão por ser um PEC, obtendo assim um padrão de interferência.

Variando o valor de Δt, e usando uma ponta de prova no centro do grid, podemos usar o valor máximo medido para aferir a estabilidade do método em função de Δt, como mostra a Tabela 1. Vemos que o método se torna instável quando a condição de CFL não é respeitada.

Tabela 1 – Valor máximo medido no centro do grid em função de Δt.

Δt	E_z Máximo (V/m)
$0.50 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.18
$0.90 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.25
$0.98 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.25
$0.99 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.24
$1.00 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	1.25
$1.01 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	$1.13 \cdot 10^{54}$
$1.10 \quad / \quad c\sqrt{(\Delta x)^{-2} + (\Delta y)^{-2}}$	$1.016 \cdot 10^{184}$

ANEXO A – CÓDIGO EXERCÍCIO 5.7

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)
f = 1e9

# --- domínio ---
min_lambda = vp / f  # comprimento de onda mínimo para 1 GHz = 3e-1 m
Nx = 100
Ny = 50
dx = dy = 0.1 * min_lambda
dt_list = [0.5, 0.9, 0.98, 0.99, 1, 1.01, 1.1] / (vp * np.sqrt((1/dx**2) + (1/dy**2)))
# dt_list = [0.99] / (vp * np.sqrt((1/dx**2) + (1/dy**2)))
max_value_per_dt = []

for dt in dt_list:
    Nt = 500
    L = Nx * dx
    T = Nt * dt

    grid_x_Ez = np.linspace(0, L, Nx + 1)
    grid_y_Ez = np.linspace(0, L, Ny + 1)
    grid_t_Ez = np.linspace(0, T, Nt + 1)

    grid_x_Hx = np.linspace(0, L, Nx + 1)
    grid_y_Hx = np.linspace(dy/2, L - dy/2, Ny)
    grid_t_Hx = np.linspace(dt/2, T - dt/2, Nt)

    grid_x_Hy = np.linspace(dx/2, L - dx/2, Nx)
    grid_y_Hy = np.linspace(0, L, Ny + 1)
    grid_t_Hy = np.linspace(dt/2, T - dt/2, Nt)

    # --- campos ---
    Ez_curr = Ez_next = np.zeros((grid_x_Ez.shape[0], grid_y_Ez.shape[0]))
    Hx_curr = Hx_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0]))
    Hy_curr = Hy_next = np.zeros((grid_x_Hy.shape[0], grid_y_Hy.shape[0]))

    # valores do tempo para salvar os snapshots e montar os heatmaps
    n_snapshots = np.linspace(5, Nt - 5, 6, dtype=int)
    Ez_snapshots = []

    # ponta de prova no centro
    probe = []
    cx, cy = Nx // 2, Ny // 2

    for n in range(Nt-1):
        # fonte onda planar
        source = np.exp(-((n * dt - 1.8e-9)**2) / (0.6e-9)**2)
        source_senoidal = np.sin(2 * np.pi * f * n * dt)

        # aplica em todos os Y em X = 0 (left boundary plane wave)
        Ez_curr[0, :] = source_senoidal
```

```
    Hx_next[:, :] = Hx_curr[:, :] - (dt/(mu*dy)) * (
        Ez_curr[:, 1:] - Ez_curr[:, :-1]
    )

    Hy_next[:, :] = Hy_curr[:, :] + (dt/(mu*dx)) * (
        Ez_curr[1:, :] - Ez_curr[:-1, :]
    )

    Ez_next[1:-1, 1:-1] = Ez_curr[1:-1, 1:-1] + (dt/eps) * (
        (Hy_next[1:, 1:-1] - Hy_next[:-1, 1:-1]) / dx -
        (Hx_next[1:-1, 1:] - Hx_next[1:-1, :-1]) / dy
    )

    # PEC an extremidade direita
    Ez_next[-1, :] = 0

    # periodic boundary condition
    Ez_next[:, 0] = Ez_next[:, Ny]
    Hx_next[:, 0] = Hx_next[:, Ny - 1]

    # atualiza os campos para o próximo passo
    Ez_curr = Ez_next.copy()
    Hx_curr = Hx_next.copy()
    Hy_curr = Hy_next.copy()

    # salva na ponta de prova
    probe.append(Ez_curr[cx, cy])

    # salva para os snapshots
    if n in n_snapshots:
        Ez_snapshots.append(Ez_curr.copy())

max_value_per_dt.append(np.max(probe))
```